

String Collection

Till now, we have discussed numbers as the standard data-types in Python. In this section of the tutorial, we will discuss the most popular data type in Python, i.e., string.

Python string is the collection of the characters surrounded by single quotes, double quotes, or triple quotes. The computer does not understand the characters; internally, it stores manipulated character as the combination of the 0's and 1's.

Each character is encoded in the ASCII or Unicode character. So we can say that Python strings are also called the collection of Unicode characters.

In Python, strings can be created by enclosing the character or the sequence of characters in the quotes. Python allows us to use single quotes, double quotes, or triple quotes to create the string.

Creating String in Python

We can create a string by enclosing the characters in single-quotes or double-quotes. Python also provides triple-quotes to represent the string, but it is generally used for multiline string or **docstrings**.

```
# Using single quotes
```

```
mystr1 = 'Hello Python'
```

```
print(mystr1)
```

```
# Using double quotes
```

```
mystr2 = "Hello Python"
```

```
print(mystr2)
```

```
# Using triple single quotes
```

```
mystr3 = '''Triple quotes are generally used for  
represent the multiline or  
docstring'''
```

```
print(mystr3)
```

```
# Using triple double quotes
mystr4 = """Triple quotes are generally used for
represent the multiline or
docstring"""
print(mystr4)
```

output:

Hello Python

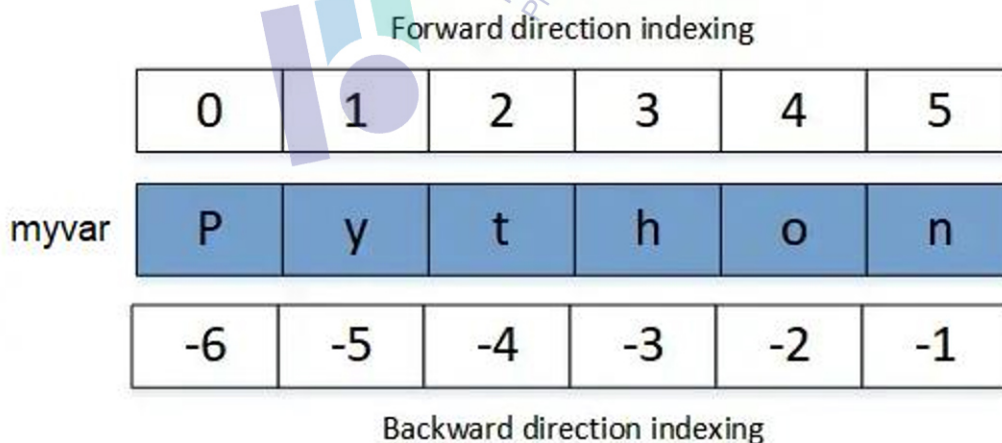
Hello Python

Triple quotes are generally used for
represent the multiline or
docstring

Triple quotes are generally used for
represent the multiline or
docstring

Strings indexing and splitting

Like other languages, the indexing of the Python strings starts from 0. For example, the string "Python" is indexed as given in the below figure.



```
myvar = "HELLO"
print(myvar[0])
print(myvar[1])
print(myvar[2])
print(myvar[3])
print(myvar[4])
# It returns the IndexError because 6th index doesn't exist
print(myvar[6])
```

output:

H
E
L
L
O

```
7 # It returns the IndexError because 6th index doesn't exist
----> 8 print(myvar[6])
      9
      10
```

IndexError: string index out of range

```
myvar = "HELLO"
print(myvar[-1])
print(myvar[-2])
print(myvar[-3])
print(myvar[-4])
print(myvar[-5])
```

output:

O

L

L

E

H

As shown in Python, the slice operator `[]` is used to access the individual characters of the string. However, we can use the `:` (colon) operator in Python to access the substring from the given string.

str = "HELLO"

H	E	L	L	O
0	1	2	3	4

str[0] = 'H' str[:] = 'HELLO'

str[1] = 'E' str[0:] = 'HELLO'

str[2] = 'L' str[:5] = 'HELLO'

str[3] = 'L' str[:3] = 'HEL'

str[4] = 'O' str[0:2] = 'HE'

str[1:4] = 'ELL'

Given String

mystr = "INTERNATIONAL"

Start 0th index to end

print(mystr[0:])

Starts 1th index to 4th index

print(mystr[1:5])

Starts 2nd index to 3rd index

```
print(mystr[2:4])
# Starts 0th to 2nd index
print(mystr[:3])
#Starts 4th to 6th index
print(mystr[4:7])
```

output:

INTERNATIONAL

INTER

TE

INT

RNA

We can do the negative slicing in the string; it starts from the rightmost character, which is indicated as -1. The second rightmost index indicates -2, and so on.

str = "HELLO"

H	E	L	L	O
-5	-4	-3	-2	-1

str[-1] = 'O' str[-3:-1] = 'LL'

str[-2] = 'L' str[-4:-1] = 'ELL'

str[-3] = 'L' str[-5:-3] = 'HE'

str[-4] = 'E' str[-4:] = 'ELLO'

str[-5] = 'H' str[::-1] = 'OLLEH'

```
mystr = 'INTERNATIONAL'
print(mystr[-1])
```

```
print(mystr[-3])
print(mystr[-2:])
print(mystr[-4:-1])
print(mystr[-7:-2])
# Reversing the given string
print(mystr[::-1])
print(mystr[-12])
```

output:

```
L
N
AL
ONA
ATION
LANOITANRETNI
N
```

Reassigning Strings

Updating the content of the strings is as easy as assigning it to a new string. The string object doesn't support item assignment i.e., A string can only be replaced with new string since its content cannot be partially replaced. Strings are immutable in Python.

```
mystr = "HELLO"
print(mystr)
mystr = "hello"
print(mystr)
```

output:

HELLO

hello

Python String Formatting Using % Operator

Python allows us to use the format specifiers used in C's printf statement. The format specifiers in Python are treated in the same way as they are treated in C. However, Python provides an additional operator %, which is used as an interface between the format specifiers and their values. In other words, we can say that it binds the format specifiers to the values.

Different printing techniques

```
myvar1=3
myvar2=6
print(myvar1,myvar2)
print('Value of myvar1:',myvar1,'and myvar2:',myvar2)
print('Value of myvar1: ' + str(myvar1)+' and myvar2: '+str(myvar2))
print('Value of myvar1: %d and myvar2: %d ' % (myvar1,myvar2))
print('Value of myvar1: {} and myvar2: {}'.format(myvar1,myvar2))
print(f'Value of myvar1: {myvar1} and myvar2: {myvar2}')
```

output:

3 6

Value of myvar1: 3 and myvar2: 6

Value of myvar1: 3 and myvar2: 6

Value of myvar1: 3 and myvar2: 6

Value of myvar1: 3 and myvar2: 6

Value of myvar1: 3 and myvar2: 6

Deleting the String:

As we know that strings are immutable. We cannot delete or remove the characters

from the string. But we can delete the entire string using the del keyword.

```
mystr = "python"
```

```
del mystr[1] #raise error due to individual string removal
```

```
#Now we are deleting entire string.
```

```
mystr1 = "python"
```

```
del mystr1
```

```
print(mystr1) #error
```

String Formatting:

Escape Sequence

In Python, **escape characters** are used inside strings to represent special characters that are hard to type directly or have a special meaning. They all start with a backslash \.

Here are the most common ones with examples:

1. Newline \n

Moves text to a new line.

```
print("Hello\nWorld")
```

Output:

Hello

World

2. Tab \t

Adds horizontal spacing (tab).

```
print("Hello\tWorld")
```

Output:
Hello World

3. Backslash \

Prints a literal backslash.

```
print("This is a backslash: \\")
```

Output:
*This is a backslash: *

4. Single Quote \'

Useful inside single-quoted strings.

```
print('It\'s Python')
```

Output:
It's Python

5. Double Quote \"

Useful inside double-quoted strings.

```
print("He said \"Hello\"")
```

Output:
He said "Hello"

6. Carriage Return \r

Moves cursor to the beginning of the line.

```
print("Hello\rWorld")
```

Output:
World

7. Backspace \b

Removes the previous character.

```
print("Hello\b\b")
```

Output:
Hello

8. Raw Strings (ignore escape characters)

Use r before a string to treat backslashes literally.

```
print("C:\newfolder\test")
```

Output:
C:
ewfolder est

What Python really sees:

- \n → newline
- \t → tab

```
print(r"C:\newfolder\test")
```

Output:
C:\newfolder\test

```
print("C:\\newfolder\\test")
```

Output:
C:\newfolder\test

There is a **line continuation trick** in Python using the backslash \

\ It ignores the new line

```
print("This \
is \
a \
trick")
```

Output:
This is a trick

String Methods

String methods in Python are **built-in functions** that you can use with **strings** to perform common tasks like searching, modifying, or formatting text.

<code>capitalize()</code>	Returns the string with first letter capitalized
<code>count()</code>	Returns the count of string or character in string
<code>startswith()</code>	Returns true if the string starts with the specified value
<code>endswith()</code>	Returns true if the string ends with the specified value
<code>find()</code>	Searches the string for a specified value and returns the position of where it was found
<code>format()</code>	Formats specified values in a string
<code>index()</code>	The <code>index()</code> method raises an exception if the value is not found.
<code>isalnum()</code>	Returns True if all characters in the string are alphanumeric
<code>isalpha()</code>	Returns True if all characters in the string are in the alphabet
<code>title()</code>	Converts the first character of each word to upper case
<code>upper()</code>	Converts a string into upper case
<code>lower()</code>	Converts a string into lower case
<code>isdigit()</code>	Returns True if given string is integer
<code>isspace()</code>	Returns True if space in string
<code>istitle()</code>	Returns True if each word in string starts with a capital letter
<code>islower()</code>	Returns True if word in string is lower case
<code>isupper()</code>	Returns True if word in string is in upper case
<code>swapcase()</code>	Swaps cases, lower case becomes upper case and vice versa
<code>join()</code>	Joins the elements of an iterable to the end of the string
<code>lstrip()</code>	Returns a left trim version of the string
<code>rstrip()</code>	Returns a right trim version of the string
<code>strip()</code>	Returns a trimmed version of the string
<code>replace()</code>	Returns a string where a specified value is replaced with a specified value
<code>split()</code>	Splits the string at the specified separator, and returns a list

capitalize()

Returns the string with first letter capitalized

```
myvar="hello, and welcome to my world."
myvar=myvar.capitalize()
print(myvar)
```

output:

Hello, and welcome to my world.

count()

Returns the count of string or character in string

```
myvar = "I love apples, apple are my favorite fruit"  
cnt = myvar.count("apple")  
print(cnt)
```

output:

2

startswith()

Returns true if the string starts with the specified value

```
txt = "Hello, welcome to my world."  
myvar = txt.startswith("Hello")  
print(myvar)
```

output:

True

endswith()

Returns true if the string ends with the specified value

```
myvar = "Hello, welcome to my world."  
out = myvar.endswith(".")  
print(out)
```

output:

True

find()

Searches the string for a specified value and returns the position of where it was found

```
txt = "Hello, welcome to my world."  
myvar = txt.find("welcome")  
print(myvar)
```

output:

7

format()

Formats specified values in a string

```
#named indexes:
txt1 = "My name is {fname}, I'm {age}".format(fname = "John",
age = 36)
#numbered indexes:
txt2 = "My name is {0}, I'm {1}".format("John",36)
#empty placeholders:
txt3 = "My name is {}, I'm {}".format("John",36)

print(txt1)
print(txt2)
print(txt3)
```

output:

```
My name is John, I'm 36
My name is John, I'm 36
My name is John, I'm 36
```

index()

The index() method raises an exception if the value is not found.

The index() method is almost the same as the find() method, the only difference is that the find() method returns -1 if the value is not found.

```
mystr = "hello world"

# Finds the position of the substring "world"
print(mystr.find("world"))

# Returns -1 when substring is not found
print(mystr.find("python"))
```

```
# Finds the position of the substring "world"
print(mystr.index("world"))

# Raises a ValueError when substring is not found
print(mystr.index("python"))
```

isalnum()

Returns True if all characters in the string are alphanumeric

```
txt = "Company12"
myvar = txt.isalnum()
print(myvar)
```

output:

True

isalpha()

Returns True if all characters in the string are in the alphabet

```
txt = "CompanyX"
myvar = txt.isalpha()
print(myvar)
```

output:

True

title()

Converts the first character of each word to upper case

```
txt = "Welcome to python"
myvar = txt.title()
print(myvar)
```

output:

Welcome To Python

upper()

Converts a string into upper case

```
txt = "Welcome to python"  
myvar = txt.upper()  
print(myvar)
```

output:

WELCOME TO PYTHON

lower()

Converts a string into lower case

```
txt = "WELCOME TO PYTHON"  
myvar = txt.lower()  
print(myvar)
```

output:

welcome to python

isdigit()

Returns True if given string is integer

```
txt = "50800"  
myvar = txt.isdigit()  
print(myvar)
```

output:

True

isspace()

Returns True if space in string

```
txt = " "
```

```
myvar = txt.isspace()  
print(myvar)
```

output:

True

istitle()

Returns True if each word in string starts with a capital letter

```
txt = "Hello, And Welcome To My World!"  
myvar = txt.istitle()  
print(myvar)
```

output:

True

islower()

Returns True if word in string is in lower case

```
txt = "hello world!"  
myvar = txt.islower()  
print(myvar)
```

output:

True

isupper()

Returns True if word in string is in upper case

```
txt = "HELLO WORLD"  
myvar = txt.isupper()  
print(myvar)
```

output:

True

swapcase()

Swaps cases, lower case becomes upper case and vice versa

```
txt = "Hello My Name Is PETER"  
myvar = txt.swapcase()  
print(myvar)
```

output:
hELLO mY nAME iS peter

join()

Joins the elements of an iterable to the end of the string

```
myTuple = ["John", "Peter", "Vicky"] # or ["John", "Peter",  
"Vicky"] or {"John", "Peter", "Vicky"} # we will learn about  
list tuple set in later classes  
myvar = ",".join(myTuple)  
print(myvar)
```

output:
John,Vicky,Peter

lstrip()

Returns a left trim version of the string

```
txt = "    banana "  
myvar = txt.lstrip() # removes the spaces at the left side of  
banana  
  
print("of all fruits", myvar, "is my favorite")
```

output:
of all fruits banana is my favorite

rstrip()

Returns a right trim version of the string

```
txt = "    banana    "  
myvar = txt.rstrip() # removes the spaces at the right side of  
banana  
print("of all fruits", myvar, "is my favorite")
```

output:

of all fruits banana is my favorite

strip()

Returns a trimmed version of the string

```
txt = "    banana    "  
myvar = txt.strip() # removes all spaces from banana  
print("of all fruits", myvar, "is my favorite")
```

output:

of all fruits banana is my favorite

replace()

Returns a string where a specified value is replaced with a specified value

```
txt = "I like bananas"  
myvar = txt.replace("bananas", "apples")  
print(myvar)
```

output:

I like apples

split()

Splits the string at the specified separator, and returns a list

```
txt = "welcome to the jungle"
myvar = txt.split()
print(myvar)#outputs a list which we will learn in the later
classes
```

output:

```
['welcome', 'to', 'the', 'jungle']
```

Loops in string

Loops in strings typically refer to using a **loop structure** (like for or while) to **iterate over the characters in a string**. Here's a quick explanation with examples in Python:

1. For loop over a string

```
text = "hello"
for i in text:
    print(i)
```

output:

```
h
e
l
l
o
```

2. Using range() and indexing

```
text = "world"
for i in range(len(text)):
    print(text[i])
```

output:

```
w
o
```

r
l
d

3. While loop with indexing

```
text = "hello world"  
i = 0  
while i < len(text):  
    print(text[i])  
    i += 1
```

output:

h
e
l
l
o

w
o
r
l
d

#count the number of uppercase letters in a string in Python using a for loop

```
text = "Hello World!"  
count = 0  
for i in text:  
    if i.isupper():  
        count += 1  
print("Uppercase letters:", count)
```

```
#count number of alphabets and digits in a string
text = "Hello World123"
```

```
alpha_count = 0
digit_count = 0
```

```
for i in text:
    if i.isalpha():
        alpha_count += 1
    elif i.isdigit():
        digit_count += 1

print("Alphabets:", alpha_count)
print("Digits:", digit_count)
```

output:

Alphabets: 10

Digits: 3



smectechnologies

II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215
GUIDING THE FUTURE WITH AI